

UAS

PENGOLAHAN CITRA DIGITAL



NAMA : Fathur Rasyid Irimawa

NIM : 202431036

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 26

ASISTEN : 1. Joshua Indra Pratama

2. Davina Najwa Ermawan

3. Izzat Islami Kagapi

4. Sakura Amastasya Salsabila Setiyanto

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.....	3
PENDAHULUAN	3
1.1 Rumusan Masalah.....	3
1.2 Tujuan Masalah.....	3
1.3 Manfaat Masalah.....	3
BAB II.....	4
LANDASAN TEORI.....	4
2.1 Konsep Dasar Pengolahan Citra Digital.....	4
2.2 Ruang Warna dan Representasi Piksel.....	4
2.2.1 Model Warna RGB.....	4
2.2.2 Model Warna HSV.....	5
2.3 Segmentasi Citra dan Thresholding Biner	5
2.4 Pendeteksian Blob dan Pengukuran Kontur.....	6
2.5 Masking, Morfologi, dan Operasi Bitwise	6
2.6 Library Pemrograman Python	6
BAB III	8
HASIL.....	8
3.1 Bukti Pengambilan Gambar	8
3.2 Deteksi Daun – Citra Asli.....	9
3.3 Deteksi Daun – Mask	10
3.4 Deteksi Daun – Segmentasi Jeruk Nipis	12
3.5 Deteksi Daun – Segmentasi Daun	13
BAB IV	16
PENUTUP.....	16
DAFTAR PUSTAKA	17

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara mengimplementasikan konversi ruang warna citra dari RGB ke HSV untuk memisahkan objek daun dan buah jeruk nipis dari latar belakang secara komputasional?
2. Bagaimana cara menentukan rentang nilai batas (*threshold*) yang presisi pada model warna HSV untuk menghasilkan citra biner (*masking*) objek yang akurat?
3. Bagaimana proses penerapan operasi pendeteksian kontur, morfologi, dan logika aljabar *bitwise* menggunakan pustaka OpenCV untuk mengekstraksi objek secara utuh dari citra aslinya?

1.2 Tujuan Masalah

1. Mengaplikasikan proses konversi model warna RGB menjadi HSV untuk mempermudah identifikasi fitur warna (hijau untuk daun dan buah jeruk nipis) yang kebal terhadap perubahan intensitas pencahayaan.
2. Menentukan nilai *lower bound* dan *upper bound* yang tepat pada ruangan warna HSV guna melakukan proses binerisasi dan membuat *mask* pemisah objek.
3. Mengeksekusi tahapan algoritma *computer vision* yang meliputi *blob detection*, penandaan batas kontur, perbaikan *noise* dengan morfologi, serta operasi *bitwise* menggunakan lingkungan pemrograman Python.

1.3 Manfaat Masalah

1. Melatih dan meningkatkan insting analitis mahasiswa dalam menyelesaikan masalah manipulasi piksel matriks skala besar. Praktikum ini memberikan pengalaman langsung dalam menulis baris kode untuk pengolahan citra menggunakan bahasa pemrograman Python.
2. Membuktikan secara empiris teori-teori fundamental pengolahan citra digital, khususnya keunggulan segmentasi berbasis ruang warna HSV dibandingkan RGB dalam mengisolasi target visual yang spesifik.
3. Logika algoritma segmentasi *masking* yang dibangun dalam modul ini dapat dijadikan kerangka dasar (*framework*) untuk pengembangan perangkat lunak berbasis deteksi visual yang lebih kompleks di masa depan, seperti sistem pendeteksi kematangan buah otomatis.

BAB II

LANDASAN TEORI

2.1 Konsep Dasar Pengolahan Citra Digital

Seiring dengan perkembangan teknologi informasi, pemanfaatan sistem visi komputer (computer vision) telah menjadi instrumen krusial dalam melakukan analisis visual secara komputasional. Pemahaman mendasar terkait sistem ini dimulai dari definisi elemen penyusun utamanya, yaitu citra. Secara fundamental, citra digital adalah gambar dua dimensi yang direpresentasikan secara numerik dengan nilai diskrit atau piksel. Piksel (picture element) tersebut berkedudukan di dalam sebuah matriks dua dimensi, di mana setiap koordinatnya mewakili tingkat intensitas cahaya pada titik spasial tertentu. Meskipun citra yang direkam oleh sensor kamera tampak mulus di mata manusia, struktur matematis aslinya hanyalah sekumpulan angka yang terikat pada aturan kuantisasi.

Kondisi citra mentah yang diambil dari perangkat keras sering kali mengalami gangguan (derau) atau ketidaksempurnaan visual yang menghambat proses ekstraksi fitur lebih lanjut. Oleh sebab itu, diperlukan suatu serangkaian tahapan matematis untuk memperbaiki kualitas tersebut. Pengolahan citra merupakan pemrosesan citra dua dimensi menggunakan komputer untuk memanipulasi dan meningkatkan kualitas gambar. Proses ini memastikan bahwa algoritma tingkat lanjut dapat dengan mudah membedakan fitur-fitur target, seperti membedakan wilayah objek dedaunan atau buah jeruk nipis dengan latar belakangnya. Proses komputasi pada citra ini mutlak diperlukan sebelum tahapan analisis lebih dalam dilakukan di lingkungan pengembangan perangkat lunak.

2.2 Ruang Warna dan Representasi Piksel

Pendekatan paling intuitif untuk membedakan objek di dalam citra digital adalah dengan menganalisis susunan spektrum warnanya. Di dalam komputasi citra, warna tidak disimpan sebagai satu entitas tunggal, melainkan dipecah ke dalam beberapa variabel yang membentuk sebuah model atau ruang warna (color space).

2.2.1 Model Warna RGB

Sistem pencitraan standar seperti kamera webcam dan layar monitor umumnya beroperasi pada model warna merah, hijau, dan biru. Model warna RGB (Red, Green, Blue) merupakan model warna dasar dalam pengolahan citra. Secara teori fisika cahaya, model warna RGB juga disebut additive color yang dapat menghasilkan warna putih jika dikombinasikan secara penuh. Di dalam arsitektur perangkat keras modern, sistem ini dialokasikan ke dalam format komputasi 8-bit. Berdasarkan kapasitas penyimpanan bit tersebut, warna RGB memiliki rentang nilai intensitas 0 hingga 255. Meskipun sederhana secara arsitektur, pendekatan RGB rentan terhadap perubahan

cahaya lingkungan, sehingga menyulitkan proses penandaan (segmentasi) objek berdasarkan persepsi visual.

2.2.2 Model Warna HSV

Untuk menanggulangi sensitivitas model warna RGB terhadap variasi pencahayaan, proses analisis warna yang membutuhkan presisi pemisahan background menggunakan alternatif model ruang warna yang lebih merepresentasikan cara penglihatan biologis. Model warna HSV terdiri dari komponen Hue, Saturation, dan Value. Setiap komponen ini mendeskripsikan secara spesifik parameter fisik cahaya. Komponen pertama, Hue menyatakan warna asli atau panjang gelombang dominan dari sebuah objek. Selain itu, Hue berfungsi untuk memisahkan warna seperti kemerahan atau kehijauan dari cahaya. Komponen kedua adalah Saturation, yang mana Saturation menyatakan tingkat kemurnian suatu warna. Secara visual, Saturation juga menunjukkan seberapa banyak warna putih diterapkan pada warna tersebut. Semakin rendah nilai saturasi, maka warna akan semakin pudar mendekati skala abu-abu.

Komponen ketiga pada ruang warna ini berkaitan langsung dengan kekuatan cahaya. Value menyatakan kecerahan warna atau jumlah cahaya yang diterima oleh mata. Dalam beberapa literatur terkait radiometri optik, Value juga dikenal dengan istilah luminansi. Transformasi matematis dari RGB menuju HSV merupakan langkah krusial sebelum implementasi pemisahan objek dilakukan. Kelebihan ruang warna HSV dibandingkan RGB adalah kemampuannya menghasilkan gradien warna objek yang lebih konsisten meskipun intensitas pencahayaannya berubah.

2.3 Segmentasi Citra dan Thresholding Biner

Inti utama dari pemisahan suatu objek yang menjadi fokus penelitian adalah proses segmentasi. Proses thresholding adalah proses pemisahan citra berdasarkan batas nilai tertentu. Dalam proses thresholding, citra warna diubah menjadi citra biner untuk membedakan objek dengan latar belakang. Output dari metode ini menghasilkan suatu matriks (masking) yang hanya berisi warna hitam dan putih. Hal ini sangat krusial dalam memetakan letak dan batasan spasial dari daun atau buah yang sedang diukur.

Metode segmentasi thresholding ini akan mengubah nilai piksel menjadi biner yaitu 0 atau 1 berdasarkan batas dari rentang suatu nilai. Kondisi batas atas (upper bound) dan batas bawah (lower bound) ini diaplikasikan pada lapisan ruang warna HSV. Algoritma program akan melakukan iterasi pada setiap piksel untuk memverifikasi apakah piksel tersebut layak dianggap sebagai wilayah objek atau bukan. Jika nilai piksel berada dalam rentang threshold, maka nilai piksel akan diubah menjadi 255, dan jika di luar rentang akan dijadikan 0. Algoritma pemisahan biner ini sangat efisien secara performa sistem. Metode thresholding sangat cocok diterapkan ketika objek memiliki perbedaan warna dengan latar belakangnya.

2.4 Pendeteksian Blob dan Pengukuran Kontur

Piksel-piksel yang telah berhasil diubah menjadi kumpulan warna putih (255) pada bidang biner perlu diidentifikasi secara berkelompok. Teknik ini dikenal dengan analisis gumpalan warna atau blob. Blob detection adalah proses mendeteksi kumpulan titik-titik piksel yang memiliki perbedaan warna (lebih gelap atau terang) dari latar belakangnya. Setelah proses deteksi, piksel-piksel tersebut kemudian disatukan ke dalam sebuah daerah atau region. Hal ini memungkinkan sistem membedakan secara logis mana kumpulan piksel yang merepresentasikan jeruk nipis dan mana yang merupakan daun.

Setelah objek disatukan, langkah selanjutnya adalah mengekstraksi parameter bentuk fisiknya. Blob analysis merupakan proses untuk mengukur properti geometri dari blob yang terdapat pada citra biner. Parameter hasil pengukuran ini lalu ditandai dengan suatu garis atau pembatas objek. Proses pencarian contour digunakan untuk mencari objek spesifik pada sebuah citra berdasarkan bentuk atau batas warnanya.

2.5 Masking, Morfologi, dan Operasi Bitwise

Hasil dari ekstraksi biner dan pencarian batas (kontur) ini lalu diimplementasikan kembali ke dalam citra berwarna asli untuk memperoleh hasil crop visual yang natural. Proses mask digunakan untuk memfokuskan pembacaan citra pada bagian tertentu yang diinginkan. Secara hierarkis, dalam mendeteksi warna, mask dibuat berdasarkan batas atas dan batas bawah rentang warna.

Untuk menempelkan topeng (mask) tersebut dengan citra utama, digunakan logika aljabar Boolean. Setelah mask dibuat, dilakukan operasi bitwise AND antara citra asli dan mask untuk menerapkan hasil segmentasi. Namun, tidak jarang hasil dari proses ini masih menyisakan piksel bintik acak yang terlepas dari bentuk utama akibat dari pemantulan cahaya yang meleset (noise). Oleh sebab itu, diperlukan operasi restoratif secara geometris. Operasi dilasi atau morfologi digunakan untuk menghilangkan derau (noise) pada citra. Dengan pendekatan ini, keseluruhan objek jeruk nipis dan dedaunan dapat ditampilkan bersih tanpa adanya gangguan dari tekstur latar belakang.

2.6 Library Pemrograman Python

Kerangka teoritis pengolahan citra digital diimplementasikan ke dalam instruksi komputer melalui modul dan pustaka yang berjalan di ekosistem pengembangan rekayasa perangkat lunak. OpenCV (Open Source Computer Vision) merupakan library untuk pemrograman teknologi computer vision. Kumpulan fungsi matematis yang ada pada modul OpenCV memudahkan proses dari mulai akuisisi kamera, konversi model warna, hingga perataan kontur tanpa perlu merakit logika iterasi dari awal.

Semua proses yang terjadi di belakang OpenCV memanipulasi rentang matriks data numerik secara intensif. NumPy (Numerical Python) adalah pustaka yang fokus

pada komputasi numerik dengan kemampuan membentuk objek N-dimensional array. Mengingat dimensi gambar praktikum yang memuat ratusan ribu piksel, efisiensi eksekusi sangat dibutuhkan oleh komputasi sistem operasi lokal. Keunggulannya adalah, struktur data array NumPy mengkonsumsi memori yang lebih kecil dan memiliki waktu eksekusi (runtime) yang lebih cepat dibandingkan tipe data list. Dengan memadukan kedua pustaka ini di atas bahasa pemrograman, eksekusi pipeline pengolahan citra mask dapat berjalan efektif secara komputasional.

BAB III

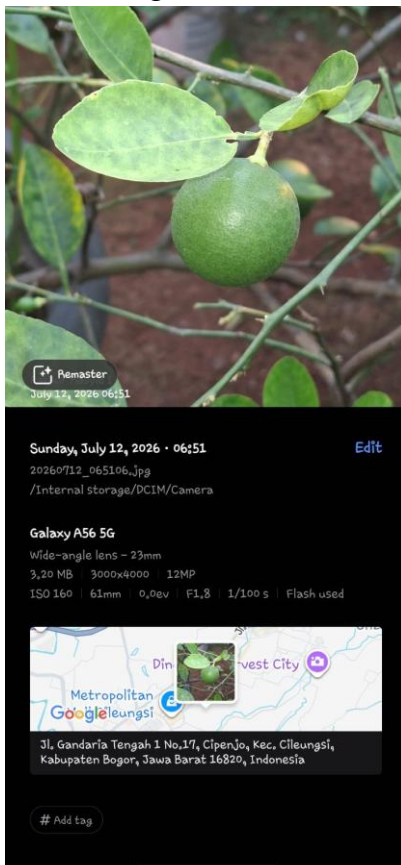
HASIL

3.1 Bukti Pengambilan Gambar

- Buah Jeruk Nipis



- Lokasi Pengambilan



3.2 Deteksi Daun – Citra Asli

- Pemanggilan Library

```
In [1]: # Fathur Rasyid Irimawa 202431036
import cv2
import matplotlib.pyplot as plt
import numpy as np
print("Library berhasil dimuat!")
```

Library berhasil dimuat!

Penjelasan code:

- Import cv2: Perintah ini berfungsi untuk memanggil pustaka OpenCV. OpenCV adalah pustaka utama yang berisi pemrograman untuk teknologi computer vision secara real-time. Pustaka ini akan mengeksekusi algoritma pengolahan citra, seperti membaca gambar jeruk nipis, mengubah ruang warna RGB ke HSV, dan melakukan operasi morfologi.
- Import matplotlib.pyplot as plt: Perintah ini memanggil modul pyplot dari pustaka Matplotlib. Fungsi utamanya untuk memplot dan menampilkan gambar hasil olahan langsung dibawah cell kode.
- Import numpy as np: Perintah ini memanggil pustaka NumPy yang berfokus pada komputasi numerik saintifik. Pustaka ini memiliki kemampuan untuk membentuk objek matriks array N-dimensi.
- Print ('...'): Perintah ini untuk mencetak teks ke layar

- Membaca dan menampilkan citra asli

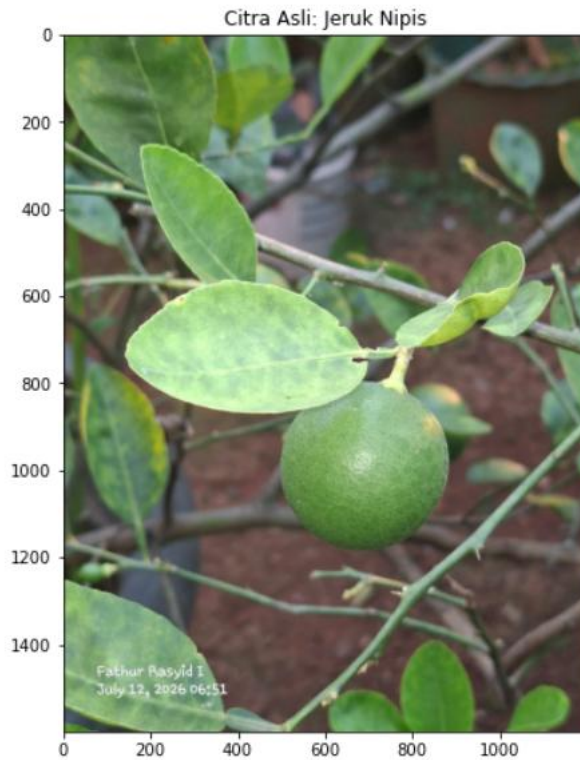
```
In [4]: # Fathur Rasyid Irimawa
img = cv2.imread('JerukNipis.jpeg')

plt.figure(figsize=(10, 8))
plt.imshow(img)
plt.title('Citra Asli: Jeruk Nipis')
plt.axis('on')
plt.show()
print(f"Resolusi matriks gambar: {img_rgb.shape}")
```

Penjelasan code:

- Img = cv2.imread.. : Membaca file gambar dari folder dan menyimpannya ke dalam variabel img
- Plt.figure.. : Mengatur ukuran kanvas gambar menjadi 10x8
- Plt.imshow: Menyiapkan data matriks untuk gambar yang akan ditampilkan
- plt.title: Menambahkan teks judul dibagian atas bingkai gambar
- plt.axis.. : Menampilkan garis ukur dan angka koordinat sumbu X dan Y di pinggiran bingkai gambar
- plt.show: Mengeksekusi seluruh instruksi plt sebelumnya dan menampilkan gambar secara visual di layar jupyter
- print(f...): Mencetak ukuran resolusi gambar

- Output



Resolusi matriks gambar: (1600, 1200, 3)

3.3 Deteksi Daun – Mask

- Pemanggilan Library

```
In [1]: # Fathur Rasyid Irimawa 202431036
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Penjelasan code:

- Import cv2: Memanggil Pustaka OpenCV
- Import numpy: Memanggil NumPy untuk manipulasi angka matriks dan rentang warna
- Import matplotlib: Memanggil Pustaka untuk menampilkan gambar pada layar jupyter

- Membaca Citra dan Konversi warna

```
In [2]: # Fathur Rasyid Irimawa 202431036
img = cv2.imread('JerukNipis.jpeg')
img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

Penjelasan code:

- Imread: Membaca file gambar dari folder penyimpanan
- BGR2HSV: Mengubah gambar ke ruang warna HSV sebagai syarat utama untuk mendeteksi warna objek dan dapat diisolasi dari intensitas Cahaya

- Pembuatan Mask

```
In [3]: # Fathur Rasyid Irimawa 202431036
lower_green = np.array([30, 40, 40])
upper_green = np.array([90, 255, 255])
mask_jeruk = cv2.inRange(img_hsv, lower_green, upper_green)
```

Penjelasan code:

- Lower: Menentukan batas bawah rentang warna hijau (gelap) dalam skala HSV
- Upper: Menentukan batas atas rentang warna hijau (terang) dalam skala HSV
- inRange: Mengeksekusi pembuatan citra biner (hitam-putih). Area yang masuk dalam rentang warna hijau akan menjadi putih dan sisanya menjadi hitam

- Menampilkan hasil akhir

```
In [4]: # Fathur rasyid Irimawa 202431036
fig, ax = plt.subplots(1, 2, figsize=(10, 5))

ax[0].imshow(img)
ax[0].set_title('Citra Asli')

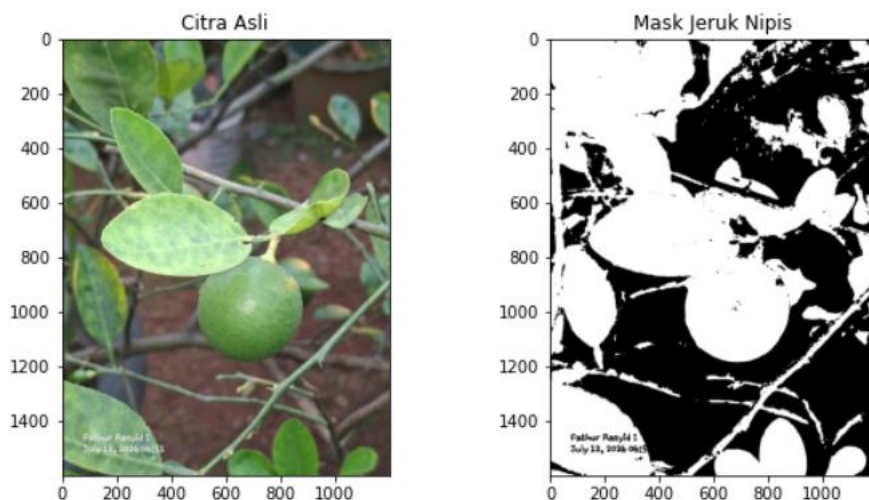
ax[1].imshow(mask_jeruk, cmap='gray')
ax[1].set_title('Mask Jeruk Nipis')

plt.show()
```

Penjelasan code:

- Subplots: Membuat kanvas yang berisi 1 baris dan 2 kolom secara sejajar
- Ax: Mengisi kolom pertama dengan gambar asli, kolom kedua dengan gambar *mask*
- Cmap: Memaksa gambar *mask* ditampilkan murni dalam format warna skala abu-abu
- Title: Menambahkan teks judul dibagian atas bingkai gambar
- Plt.show: Memunculkan seluruh kanvas gambar ke layar.

- Output



3.4 Deteksi Daun – Segmentasi Jeruk Nipis

- Pemanggilan Library

```
In [1]: # Fathur Rasyid Irimawa 202431036
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Penjelasan code:

- Cv2: Memanggil Pustaka OpenCV
- NumPy: Memanggil NumPy untuk manipulasi angka matriks dan rentang warna
- Matplotlib: Memanggil Pustaka untuk menampilkan gambar pada layar jupyter

- Membaca Citra dan Konversi warna

```
In [2]: # Fathur Rasyid Irimawa 202431036
img = cv2.imread('JerukNipis.jpeg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

Penjelasan code:

- Imread: Membaca file gambar dari folder penyimpanan
- BGR2RGB: Mengonversi ruang warna citra dari BGR menjadi RGB agar warna yang ditampilkan tidak tertukar.
- BGR2HSV: Mengubah gambar ke ruang warna HSV sebagai syarat utama untuk mendeteksi warna objek dan dapat diisolasi dari intensitas Cahaya

- Eksekusi segmentasi buah jeruk nipis

```
In [3]: # Fathur Rasyid Irimawa
lower_jeruk = np.array([35, 50, 50])
upper_jeruk = np.array([85, 255, 255])
mask_jeruk = cv2.inRange(img_hsv, lower_jeruk, upper_jeruk)
segmentasi_jeruk = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_jeruk)
```

Penjelasan code:

- Lower: Menetapkan nilai batas bawah rentang warna spesifik pada buah jeruk nipis dalam skala matriks HSV
- Upper: Menetapkan nilai batas atas rentang warna spesifik pada buah jeruk nipis dalam skala matriks HSV
- Mask: Membuat matriks pembatas biner (hitam-putih) secara background berdasarkan rentang nilai HSV yang sudah ditetapkan sebelumnya
- Segmentasi: Menggabungkan gambar RGB asli dengan matriks pembatas (mask). Pixel warna di luar area buah jeruk nipis akan otomatis diubah menjadi hitam

- Menampilkan hasil segmentasi

```
In [4]: # Fathur Rasyid Irimawa
fig, ax = plt.subplots(1, 2, figsize=(10, 5))

ax[0].imshow(img)
ax[0].set_title('Citra Asli')

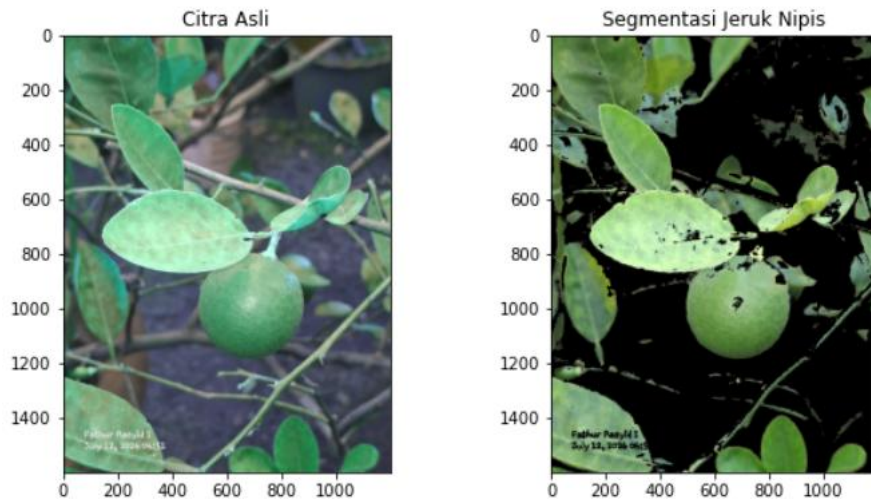
ax[1].imshow(segmentasi_jeruk)
ax[1].set_title('Segmentasi Jeruk Nipis')

plt.show()
```

Penjelasan code:

- Subplots: Membuat kanvas yang berisi 1 baris dan 2 kolom secara sejajar
- Ax: Mengisi kolom pertama dengan gambar asli, kolom kedua dengan gambar segmentasi jeruk nipis
- Title: Menambahkan teks judul dibagian atas bingkai gambar
- Plt.show: Memunculkan seluruh kanvas gambar ke layar.

- Output



3.5 Deteksi Daun – Segmentasi Daun

- Pemanggilan Library

```
In [1]: # Fathur Rasyid Irimawa 202431036
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Penjelasan code:

- Cv2: Memanggil Pustaka OpenCV
- NumPy: Memanggil NumPy untuk manipulasi angka matriks dan rentang warna
- Matplotlib: Memanggil Pustaka untuk menampilkan gambar pada layar jupyter

- Membaca citra dan konversi warna

```
In [2]: # Fathur Rasyid Irimawa
img = cv2.imread('JerukNipis.jpeg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

Penjelasan code:

- Imread: Membaca file gambar dari folder penyimpanan
- BGR2RGB: Mengonversi ruang warna citra dari BGR menjadi RGB agar warna yang ditampilkan tidak tertukar.
- BGR2HSV: Mengubah gambar ke ruang warna HSV sebagai syarat utama untuk mendeteksi warna objek dan dapat diisolasi dari intensitas Cahaya

- Eksekusi Segmentasi

```
In [3]: # Fathur Rasyid Irimawa 202431036
lower_hijau = np.array([25, 30, 30])
upper_hijau = np.array([90, 255, 255])
mask_awal = cv2.inRange(img_hsv, lower_hijau, upper_hijau)

kernel = np.ones((5, 5), np.uint8)
mask_bersih = cv2.morphologyEx(mask_awal, cv2.MORPH_CLOSE, kernel)

segmentasi_hijau = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_bersih)
```

Penjelasan code:

- Lower: Menetapkan nilai batas bawah rentang warna spesifik pada buah jeruk nipis dalam skala matriks HSV
- Upper: Menetapkan nilai batas atas rentang warna spesifik pada buah jeruk nipis dalam skala matriks HSV
- Mask_awal: Membuat citra biner (hitam-putih) awal berdasarkan rentang warna hijau tersebut.
- Kernel: Membuat matriks sapuan berukuran 5x5 piksel yang digunakan sebagai alat perata.
- Mask_bersih: Menerapkan operasi morfologi closing untuk menambal titik-titik hitam (lubang) yang ada di dalam area putih daun.
- Segmentasi_hijau: Menggabungkan gambar asli dengan matriks mask yang sudah rapi, membuang latar belakang menjadi hitam.

- Menampilkan hasil segmentasi

```
In [4]: # Fathur Rasyid Irimawa 202431036
fig, ax = plt.subplots(1, 2, figsize=(10, 6))

ax[0].imshow(img)
ax[0].set_title('Citra Asli')

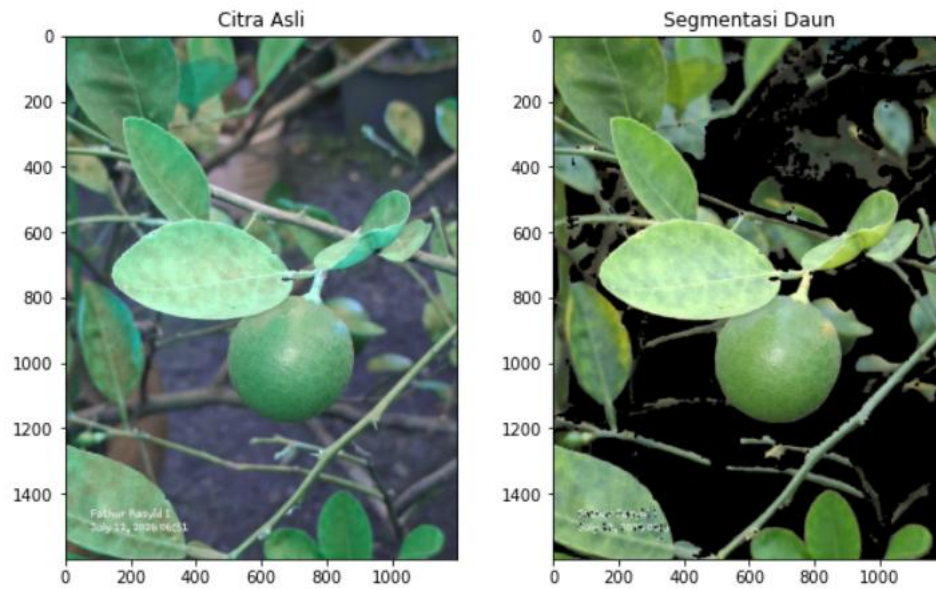
ax[1].imshow(segmentasi_hijau)
ax[1].set_title('Segmentasi Daun')

plt.show()
```

Penjelasan code:

- Subplots: Membuat kanvas yang berisi 1 baris dan 2 kolom secara sejajar
- Ax: Mengisi kolom pertama dengan gambar asli, kolom kedua dengan gambar segmentasi jeruk nipis
- Title: Menambahkan teks judul dibagian atas bingkai gambar
- Plt.show: Memunculkan seluruh kanvas gambar ke layar.

- Output



- Kenapa Output Segmentasi Daun dan Buah Mirip?
Karena kedua objek memiliki warna yang sama

BAB IV

PENUTUP

1. Efektivitas model warna HSV: Konversi ruang warna RGB menjadi HSV merupakan Langkah awal dalam proses segmentasi warna objek. Dan ini terbukti karena model HSV dapat memisahkan informasi warna asli dari intensitas Cahaya sehingga gradien warna objek yang dideteksi tetap konsisten walaupun ada gangguan pantulan Cahaya atau bayangan pada citra asli.
2. Keberhasilan segmentasi thresholding: pemisahan objek daun dan buah jeruk nipis dapat dieksekusi dengan baik dengan metode segmentasi thresholding. Dengan menetapkan nilai batas bawah dan atas pada rentang hijau, algoritma berhasil membedakan area buah dan daun (255) dengan area latar belakang (0) dalam bentuk citra biner mask.
3. Keterbatasan segmentasi warna Tunggal: hasil dari pengujian pada citra jeruk nipis, segmentasi berbasis rentang warna memiliki suatu Batasan ketika terdapat lebih dari satu objek dengan warna yang identic. Hal ini menyebabkan algoritma menangkap dua objek yaitu buah jeruk nipis dan daunnya secara bersamaan dalam satu mask yang sama.
4. Signifikansi operasi morfologi: adanya pantulan Cahaya matahari pada permukaan daun menyebabkan munculnya noise pada hasil masking awal. Penerapan operasi morfologi terbukti efektif dalam menambal noise, sehingga hasil akhir segmentasi daun menggunakan operasi logika bitwise AND tampak solid, utuh dan rapih.
5. Efisiensi Pustaka pemrograman: penggunaan Bahasa pemrograman Python yang dikombinasikan dengan Pustaka OpenCV dan NumPy terlihat sangat efisien untuk operasi computer vision. NumPy mampu menangani beban komputasi matriks piksel dengan skala yang besar dengan konsumsi memori yang ringan dan waktu prosesnya cepat, Adapun OpenCV memudahkan pemanggilan fungsi kompleks menjadi suatu instruksi yang sederhana.

DAFTAR PUSTAKA

- [1] Adianto, W., Putra, E. D., Noprisson, H., Ayumi, V., Utami, M., & Purba, M. (2024). Implementasi Metode Color Blob Detection Pada Objek Daun Sawi. *JCOSIS (Journal Computer Science and Information System)*, 1(1), 20-26.
<https://jurnal.imsi.or.id/index.php/jcosis/article/view/176/139>
- [2] Akbar, M. F. M., Fitriyah, H., & Akbar, S. R. (2023). Analisis Color Space untuk Spesifikasi Perancangan Perangkat Lunak pada Embedded System Deteksi Penyakit Busuk Selada. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, 7(5), 2186-2193.
<https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/12667/5762>
- [3] Goenawan, A. D., Rachman, M. B. A., & Pulungan, M. P. (2022). Identifikasi Warna Pada Objek Citra Digital Secara Real Time Menggunakan Pengolahan Model Warna HSV. *Seminar Nasional Mahasiswa Ilmu Komputer dan Aplikasinya (SENAMIKA)*, Jakarta. 3(1) 280-281.
<https://conference.upnvj.ac.id/index.php/senamika/article/view/2011/1588>
- [4] Setyo, A. S., Fitriyah, H., & Primananda, R. (2023). Sistem Deteksi Tip Burn pada Selada Hidroponik menggunakan Metode Thresholding pada Warna Hue Saturation Value. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, 7(3), 1511-1517.
<https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/12515/5686>